CSS & HTML COMPREHENSIVE EXERCISE

Build *TechPulse* Magazine

A full-page website project that forces you to use every important CSS concept — from the box model and Flexbox to transitions, pseudo-classes, and responsive design. One project. Everything you've learned. All at once.

- HTML + CSS only
- 6 sections to complete

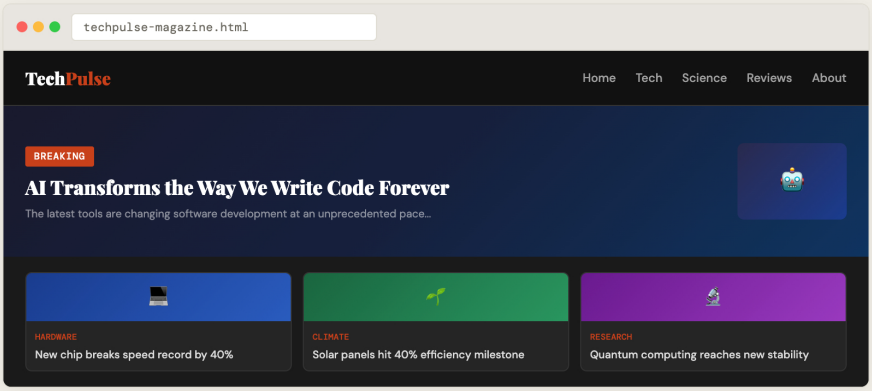
PROJECT TYPE	FILES REQUIRED	DELIVERABLE
Website build	2 files min	index.html + style.css

THE BRIEF

What you are building

You are building the homepage of TechPulse — a fictional technology news magazine. The design is already decided for you below. Your job is to produce clean, well-structured HTML and write the CSS that makes it look exactly like the specification. Every section of the page targets a specific set of CSS skills.

FINAL RESULT PREVIEW



SKILLS TESTED

CSS concepts you will practise

Every concept below appears in at least one task. Required concepts must appear in your submission. Bonus concepts earn extra marks.

01
Box Model
padding, margin, border, box-sizing. Spacing every element correctly.
Required

02
Flexbox
display:flex, justify-content, align-items, gap, flex-wrap.
Required

03
Selectors
Element, class, ID, descendant, pseudo-class selectors.
Required

04

05

06

Typography

font-family, font-size, font-weight, line-height, letter-spacing, text-transform.

Required

Colours

Hex, RGB, named colours. background-color vs color. Opacity.

Required

Pseudo-classes

:hover, :focus, :active, :nth-child, :first-child, :last-child.

Required

07

Transitions

transition property — smooth hover effects on links, buttons and cards.

Required

08

CSS Variables

:root { --variable: value } for consistent colour and spacing across the whole file.

Required

09

Navigation Bar

nav, ul, li, a. Horizontal layout. Active link. Hover highlight.

Required

10

CSS Grid

display:grid for the article card layout. grid-template-columns, gap.

Required

11

@media queries

Responsive breakpoints. Layout changes at mobile width.

Bonus

12

@keyframes

A CSS animation on at least one element — badge pulse, image slide, or fade-in.

Bonus

TASKS

Six tasks to complete

Complete the tasks in order. Each one builds on the previous. The marks shown are for each task. Read the specification inside each task carefully — it tells you exactly what CSS to write.

01

Project setup + CSS Variables + Typography

Box model reset, CSS variables, Google Font, base styles

[20 marks]

- [ ] Create `index.html` and `style.css`. Link them correctly. Add the viewport meta tag and UTF-8 charset.
- [ ] At the top of your CSS file, declare at least 5 CSS custom properties inside `:root { }`. You must include variables for: a primary background colour, a primary text colour, an accent colour, a font stack, and a base spacing unit. Use these variables throughout — never repeat a raw colour value.
- [ ] Apply a box model reset: `* { box-sizing: border-box; margin: 0; padding: 0; }`
- [ ] Link a Google Font. Apply it to the `body` via your CSS variable. Set a comfortable `font-size`, `line-height`, and `color` on the body.
- [ ] Add your page content (the heading "TechPulse") using a semantic `<header>` element, not a bare `<div>`. Use the correct semantic tags throughout: `<nav>`, `<main>`, `<section>`, `<article>`, `<footer>`.

REQUIRED CSS VARIABLES

```
:root {
  --color-bg:       your value ;
  --color-text:     your value ;
  --color-accent:   your value ;
  --font-main:      your font ;
  --spacing-unit:   e.g. 8px ;
  /* Add more as you need them */
}
```

02

Navigation Bar

Flexbox layout, hover transitions, active state, fixed position

[15 marks]

- [ ] Build the navigation using `<nav> > <ul> > <li> > <a>`. Include links: Home, Technology, Science, Reviews, About.
- [ ] Use Flexbox to place the TechPulse logo on the left and the nav links on the right. Use `justify-content: space-between` and `align-items: center`.
- [ ] Remove all list bullet points and link underlines. Set link text colour using your CSS variable.
- [ ] Add a `transition` to all links. On `:hover`, change the background colour smoothly. The transition must use `ease` timing and last at least `0.2s`.
- [ ] Give exactly one link the class `active`. Style `a.active` to look visually different from other links — different background, border-bottom, or colour.
- [ ] Make the navbar sticky — it must stay at the top when the user scrolls. Use `position: sticky; top: 0;`.

03

Hero Section

[15 marks]

Background, Flexbox two-column layout, box model, image styling

- [ ] Create a full-width **hero section** with a dark background colour. Inside it, use **Flexbox** to create a two-column layout: text on the left, an image or coloured box on the right. Use `gap` for spacing between them.
- [ ] The hero must have generous `padding` (at least 48px top and bottom). No content should touch the edges.
- [ ] Include a "Breaking" label badge. Style it using `display: inline-block`, `padding`, `border-radius`, and your accent colour as the background.
- [ ] The headline must use a large `font-size` (at least `2rem`), bold `font-weight`, and a carefully chosen `line-height` (between 1.1 and 1.3 for headlines).
- [ ] The image placeholder must use a set `width` and `height`, have `border-radius` applied, and use `object-fit: cover` if using a real `<img>` tag.

FLEXBOX TWO-COLUMN HERO PATTERN

```
.hero {
  display: flex;
  align-items: center;
  gap: 40px;
  padding: 64px 40px;
  background-color: var(--color-bg-dark);
}

.hero-text {
  flex: 1; /* takes remaining space */
}

.hero-image {
  width: 320px;
  height: 220px;
  border-radius: 8px;
  object-fit: cover;
  flex-shrink: 0;
}
```

04 Article Card Grid

CSS Grid, card design, :hover effects, box-shadow, overflow

[20 marks]

- [ ] Create a section with the heading "Latest Articles". Below it, build a grid of **at least 6 article cards** using `display: grid` and `grid-template-columns: repeat(3, 1fr)`. Use `gap` for spacing.
- [ ] Each card must contain: a coloured image area (or real `<img>`), a category label, a headline, a short description, and a "Read more" link. Use `<article>` tags for each card.
- [ ] Style the cards with a white background, `border-radius`, and a subtle `box-shadow`. The image area must use `overflow: hidden` so nothing spills outside the rounded corners.
- [ ] On `:hover`, cards must **lift** using `transform: translateY(-6px)` and deepen their `box-shadow`. Both changes must use a `transition` for smooth movement.
- [ ] Use `:nth-child` to give every **first card in a row** (cards 1 and 4) a different accent colour for their category label. Use at least 3 different category colours across all cards.
- [ ] The "Read more" link must use the `:hover` pseudo-class to change colour and show an underline *only on hover* (not by default). Use `text-decoration: none` by default and `text-decoration: underline` on hover.

CARD HOVER LIFT PATTERN

```
.card {
  transition: transform 0.25s ease, box-shadow 0.25s ease;
  box-shadow: 0 2px 8px rgba(0,0,0,0.08);
}

.card:hover {
  transform: translateY(-6px);
  box-shadow: 0 16px 32px rgba(0,0,0,0.14);
}
```

05 Sidebar + Newsletter Form

Two-column page layout, form styling, :focus states, :valid/:invalid

[20 marks]

- [ ] Create a two-column layout below the article grid using **Flexbox**: a wide main column (about 65%) containing a "Featured Story" section, and a narrower sidebar (about 32%) on the right. The sidebar must not collapse — use `flex-shrink: 0`.
- [ ] Inside the sidebar, build a **Newsletter sign-up form** with: a name input, an email input, and a submit button. Use correct `<label for="">` connections and semantic HTML.
- [ ] Style all `input` elements with a visible `border`, `border-radius`, `padding`, and `width: 100%`. Use `outline: none` to remove the default browser focus ring.
- [ ] Add a custom `:focus` style on inputs: change the `border-color` to your accent colour and add a faint `box-shadow`

glow. The change must use a `transition`.

- [ ] Use `input:valid` and `input:invalid:not(:placeholder-shown)` to show a green border when the email is valid and a red border when it is not. Do not show a red border on an empty field.
- [ ] The submit button must be full-width (`width: 100%`), use your accent colour as its background, and have a clear `:hover` state that darkens the background and uses a `transition`. The button must also scale down slightly on `:active` using `transform: scale(0.97)`.

06 Footer + Final Polish

Flexbox footer, CSS table, horizontal rule, :last-child

[10 marks]

- [ ] Build a `<footer>` with a dark background. Inside it, use Flexbox with three columns: site branding on the left, a list of quick links in the centre, and social links on the right. Use `justify-content: space-between`.
- [ ] Add a simple **data table** somewhere on the page (e.g. "Top Articles This Week" with columns: Rank, Title, Views). Style it with a header row using your dark background colour and white text, alternating row colours using `:nth-child(even)`, and a bottom border on each row using `border-bottom`. Use `:last-child` to remove the border from the final row.
- [ ] Add a visible `<hr>` divider styled with CSS — change its `border`, `height`, and colour. Do not use the default browser style.
- [ ] Review your whole page. Every `padding` and `margin` must use your `--spacing-unit` CSS variable or a multiple of it. No raw pixel values for spacing.

EXTRA CREDIT

Bonus tasks (+15 marks available)

Complete these only after all 6 main tasks are done.

📱

Responsive Design +8 marks

Add `@media (max-width: 768px)` rules. The 3-column card grid must collapse to 1 column. The hero two-column layout must stack. The navbar links must collapse or wrap. The sidebar must move below the main content.

✨

CSS Animation +4 marks

Write a `@keyframes` animation. Options: a pulsing "Live" badge on the hero, a fade-and-slide entrance for the cards on page load, or a spinning loading indicator. Must use `animation-duration`, `animation-timing-function`, and `animation-iteration-count`.

🎨

Dark Mode +3 marks

Use `@media (prefers-color-scheme: dark)` to redefine your CSS variables so the entire site switches to a dark theme automatically. Change backgrounds, text colours, and borders — nothing should be unreadable in dark mode.

🔗

Multi-page site +ungraded

Create a second page — `about.html` — with a different layout but the same `style.css`. Demonstrate that your CSS variables and shared styles work across multiple HTML files without duplication.

EXTRA CREDIT

Bonus tasks (+15 marks available)

These tasks are optional but will help you master advanced CSS. Follow the hints and examples — you are not expected to be perfect.

📱

Responsive Design +8 marks

Use `@media (max-width: 768px)` to make your layout mobile-friendly.

HINTS

- 3-column grid → 1 column
- Hero section → stack vertically
- Sidebar → move below main content

```
@media (max-width: 768px) {
  .cards { grid-template-columns: 1fr; }
  .hero { flex-direction: column; }
```

✨

CSS Animation +4 marks

Use `@keyframes` to animate an element like a badge or card.

HINT

Define animation first, then apply it using `animation` property.

```
@keyframes pulse {
  0% { transform: scale(1); }
  50% { transform: scale(1.1); }
  100% { transform: scale(1); }
}
```

```
  .layout { flex-direction: column; }
}
```

```
.badge {
  animation: pulse 1.5s ease infinite;
}
```

🎨

Dark Mode +3 marks

Use `@media (prefers-color-scheme: dark)` and update your CSS variables.

```
:root {
  --color-bg: #ffffff;
  --color-text: #000000;
}

@media (prefers-color-scheme: dark) {
  :root {
    --color-bg: #0d0d0d;
    --color-text: #ffffff;
  }
}
```

🔗

Multi-page site Practice

Create a second page (`about.html`) and reuse your existing CSS file.

HINT

Do NOT copy styles — link the same `style.css` file.

```
<link rel="stylesheet" href="style.css">
```

💬 **Important:** Bonus tasks are about experimenting. Even if your solution is not perfect, you can still earn marks if you correctly apply the CSS concepts.

PROJECT STRUCTURE

How to organise your files

Keep your files in one folder. Name them exactly as shown below.

```
techpulse/
  ├── index.html    ← your main HTML file
  ├── style.css     ← ALL your CSS goes here
  └── images/       ← optional: any images you use
```

One CSS file rule. All your styles must be in `style.css` . No inline styles (`style=""` attribute on HTML tags). No `<style>` blocks in the HTML. This is how real projects work — HTML is structure, CSS file is style.

DESIGN SPEC

Suggested colour palette & fonts

You may use these colours or choose your own. Whatever you choose, declare them as CSS variables.

Dark background #0d0d0d · Page background #faf9f6 · Accent red #c8401a · Tech blue #1a3c8f · Science green #1a6640 · Body text #6b6860

ADVICE

Tips before you start

🧱

Structure first, style second

Write all your HTML for a section before writing any CSS for it. A well-structured HTML page is easier to style. If your CSS isn't working, check the HTML structure first.

🔍

Use DevTools constantly

Right-click any element → Inspect. Use the box model visualiser to check padding and margin. Live-edit CSS in the browser before committing to your file. Every browser has this built in.

📦

Variables eliminate errors

If you define `--color-accent: #c8401a` once and use `var(--color-accent)` everywhere, you can change the whole site's accent colour in one line. Repeat a raw hex 20 times and you'll have 20 places to edit.

📐

Flexbox on the parent, not the children

`display: flex`, `justify-content`, and `align-items` go on the **container**. The children just respond. This is the most common mistake — putting flex properties on the wrong element.